IAE Project

Crescenzi and Takes-Kosters Algorithm

Ayush Kanani – 2024101049
Dhyey Thummar – 2024101024

May 6, 2026

Abstract

This report studies the graph diameter problem and compares two practical diameter algorithms: Crescenzi et al.'s iFUB method and the Takes-Kosters BoundingDiameters algorithm. Both algorithms were implemented in Python and evaluated on one real-world Facebook graph and several synthetic graph families, including Erdős-Rényi, Barabási-Albert, and Watts-Strogatz graphs of varying sizes. We compare the algorithms using runtime, number of breadth-first search traversals, and accuracy against an exact baseline on smaller graphs. The goal of the study is to understand how graph structure affects practical performance and scalability.

1 Introduction: The Problem

The *diameter* of an undirected, unweighted graph $G = (V, E)$ is the maximum shortest-path distance between any pair of vertices:

$$\text{diam}(G) = \max_{u,v \in V} d(u, v).$$

Computing the diameter is a fundamental graph problem because it captures the largest communication distance in a network and is useful in the analysis of social networks, communication graphs, and large sparse systems.

The most direct exact approach is based on the all-pairs shortest path (APSP) idea: compute the shortest path distance from every node to every other node, and then take the maximum over all pairs. In an unweighted graph, this can be done by running breadth-first search (BFS) from each vertex, giving a time cost of $O(n(n + m))$, where $n = |V|$ and $m = |E|$. For large sparse graphs, this is often too expensive in practice because it requires a BFS from every node. For this reason, practical exact algorithms aim to avoid exploring every vertex as a BFS source. In this project, we study two such methods that are designed to work well on sparse real-world networks: Crescenzi et al.'s iFUB algorithm and the Takes-Kosters BoundingDiameters algorithm. Our goal is to compare these methods both conceptually and experimentally, and to understand how graph family and graph size affect their runtime and BFS usage in practice.

2 Algorithms Studied

2.1 Crescenzi: iFUB with 4-Sweep High-Degree Initialization

The iFUB (*iterative Fringe Upper Bound*) algorithm improves practical diameter computation by first finding a strong lower bound using BFS sweeps and then refining upper and lower bounds layer by layer from a central candidate vertex. In this project, the starting node for the 4-Sweep procedure is chosen as a high-degree vertex, which often gives a stronger initial lower bound in practice.

Theoretical Complexity. Each BFS costs $O(n+m)$. The practical efficiency of iFUB comes from requiring only a limited number of BFS traversals on many real-world sparse graphs. However, in the worst case, the method may still perform many BFS traversals, leading to a worst-case running time of $O(n(n+m))$. The main intuition is to first locate a center-like starting vertex using 4-Sweep, then run BFS from that vertex and inspect its fringe layers from the outside inward. At each layer, the algorithm computes eccentricities only where needed and maintains lower and upper bounds on the graph diameter. Once the two bounds coincide, the exact diameter is known.

Algorithm Outline.

1. Choose a high-degree start vertex r_1.
2. Run BFS from r_1 to get a farthest node a_1.
3. Run BFS from a_1 to get a farthest node b_1.
4. Set r_2 as the middle vertex of a shortest path from a_1 to b_1.
5. Run BFS from r_2 to get a farthest node a_2.
6. Run BFS from a_2 to get a farthest node b_2.
7. Set u as the middle vertex of a shortest path from a_2 to b_2.
8. Initialize a lower bound using the best sweep eccentricity.
9. Run BFS from u to get level sets around u.
10. Inspect fringe layers from the outside inward.
11. For each node in the current fringe, compute eccentricity if needed.
12. Update lower and upper bounds on the diameter.
13. Stop when the lower bound equals the upper bound.

2.2 Takes-Kosters: BoundingDiameters

The Takes-Kosters algorithm maintains lower and upper bounds on the eccentricities of candidate vertices and uses these bounds to prune vertices that cannot improve the current diameter estimate. In our implementation, the candidate selection strategy alternates between promising upper-bound and lower-bound choices, with degree-based tie-breaking.

Theoretical Complexity. As with iFUB, each BFS costs $O(n+m)$. The algorithm can terminate much earlier than the naive baseline when bounds tighten quickly, but in the worst case it may still require many BFS traversals. Therefore, its worst-case running time is also $O(n(n+m))$. The algorithm starts with all vertices as candidates and repeatedly selects a candidate vertex from which it runs BFS. The distances returned by that BFS are used to tighten lower and upper bounds on the eccentricity of every candidate, as well as global bounds on the graph diameter. Vertices that can no longer improve the answer are pruned, which can greatly reduce the total number of BFS traversals in practice.

Algorithm Outline.

1. Initialize all vertices as candidates.
2. Initialize eccentricity lower bounds and upper bounds for each candidate.
3. Initialize global diameter lower and upper bounds.
4. Repeatedly select a candidate using the bound-based selection rule.
5. Run BFS from the selected candidate and compute its eccentricity.
6. Update the global diameter lower and upper bounds.
7. Update eccentricity bounds of all candidates using the BFS distances.
8. Remove candidates that can no longer improve the answer.
9. Stop when no useful candidate remains or the global bounds meet.

3 Implementation Details

This project was implemented entirely in Python. The code is organized into separate modules for algorithms, graph loading, graph generation, experiment management, and plotting.

3.1 Graph Loading and Facebook Ego-Network Merging

The Facebook data in our repository follows the SNAP ego-network format. Each `.edges` file corresponds to one ego network and lists connections among the observed alter nodes of a single ego user. Our loader constructs one merged undirected graph by adding all alter-to-alter edges, inserting the ego node for each network, and connecting that ego node to the alters observed in its local network. If the merged graph were disconnected, our pipeline would keep the largest connected component for fair diameter benchmarking. In our experiments, the resulting `facebook_combined` graph has 4039 nodes and 88,234 edges.

3.2 Synthetic Graph Generation

We use three synthetic graph families to study how the algorithms react to different structural properties. Erdős-Rényi graphs represent nearly uniform random connectivity, Barabási-Albert graphs represent scale-free networks with hubs, and Watts-Strogatz graphs represent small-world networks with local clustering and short path lengths. For the main benchmark profile, we generated connected graphs with 1000, 2000, 5000, 10,000, and 20,000 nodes for each family.

3.3 Other Implementation Details

Both algorithms were implemented in Python using adjacency-list graph representations. We use a separate exact baseline only for smaller graphs, where running BFS from every vertex is still affordable. For each benchmark run, we record runtime, BFS traversals, and bound information. The Crescenzi implementation follows the iFUB workflow with 4-Sweep high-degree initialization, while the Takes-Kosters implementation follows the BoundingDiameters bound-pruning strategy with alternating candidate selection.

4 Experimental Setup

4.1 Datasets

The experiments use one real-world graph and multiple synthetic graph families:

- **Facebook Combined:** merged ego-network graph constructed from the provided Facebook dataset.
- **Erdős-Rényi:** random graphs used to study behavior on approximately uniform random structure.
- **Barabási-Albert:** scale-free graphs used to study behavior under hub-dominated structure.
- **Watts-Strogatz:** small-world graphs used to study behavior on locally clustered graphs with short path lengths.

The final report uses the `large` benchmark profile. This profile includes synthetic graph sizes 1000, 2000, 5000, 10,000, and 20,000 for each family, along with the merged Facebook graph. Exact-baseline validation was performed earlier on smaller benchmark profiles containing 120- and 220-node graphs.

4.2 Metrics Tracked

The following metrics were recorded during experiments:

- runtime in seconds,
- number of BFS traversals,
- returned diameter,
- exact diameter on smaller graphs,
- absolute error,
- relative error,
- lower bound and upper bound where relevant.

We also recorded memory usage in the benchmark CSV, although memory is treated as a secondary metric in this report because the clearest differences between the two algorithms appeared in runtime and BFS counts.

4.3 Experimental Procedure

Both algorithms were executed on exactly the same graph instances generated by our benchmark scripts. The results were exported automatically to a CSV file, and the plots in this report were produced directly from that CSV using a separate plotting script. This ensured that the textual analysis and the figures were based on the same recorded benchmark output.

5 Results and Comparative Analysis

5.1 Comparison with the Exact Baseline

The current report is based on the `large` benchmark profile, so the CSV contains large synthetic graphs and the `facebook_combined` graph, but no exact-baseline rows. Therefore, this run is used mainly for scalability analysis. In earlier smaller-profile runs, both algorithms matched the exact diameter on all validated synthetic graphs, so the present comparison should be read as a performance comparison between already validated implementations.

5.2 Speed vs. Quality Tradeoff

In the current large-profile run, the main tradeoff is computational cost, not answer quality. The recorded lower and upper bounds coincide in the benchmark output, so the comparison is dominated by runtime and BFS count.

On the synthetic benchmark set, Takes-Kosters was clearly faster overall. Crescenzi required about 534.35 seconds in total, while Takes-Kosters required about 89.96 seconds. The BFS counts show the same trend: Crescenzi performed 82,432 BFS traversals, whereas Takes-Kosters performed 12,399. Thus, Takes-Kosters achieved the same practical goal using far fewer BFS explorations on large synthetic graphs.

5.3 Effect of Graph Structure

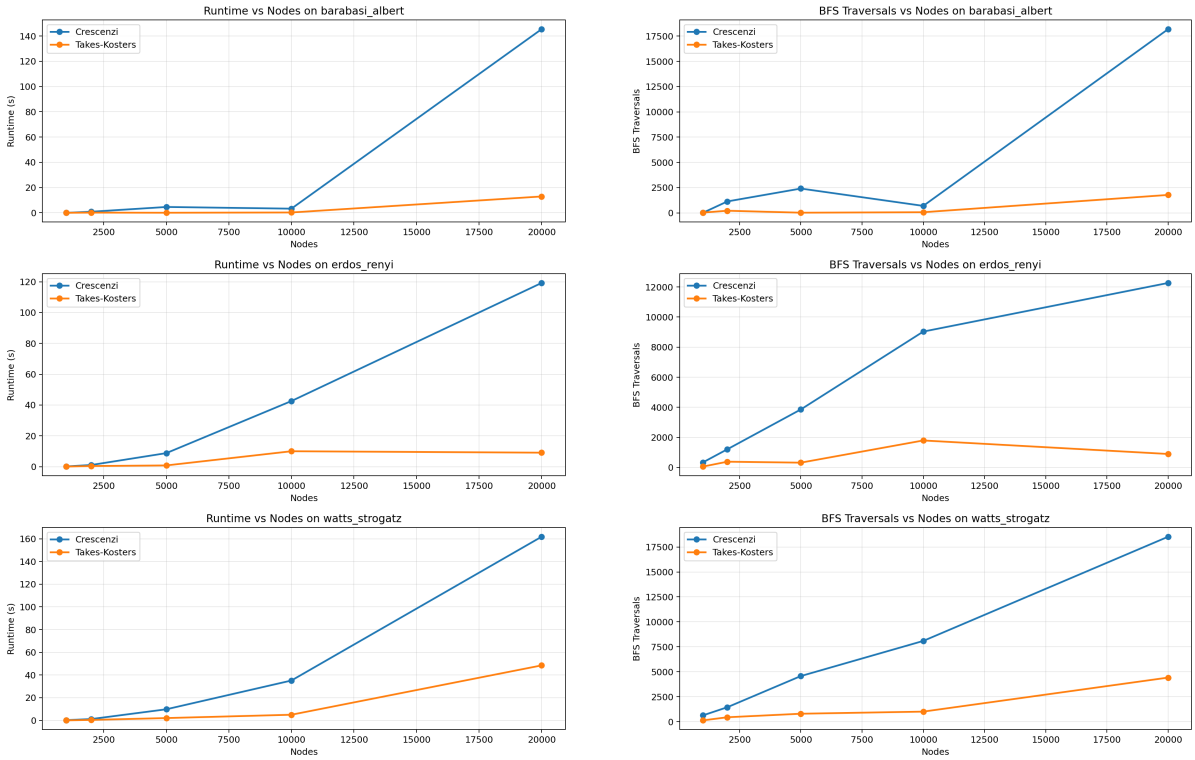
Graph structure had a clear effect on performance. Takes-Kosters was faster on 4/5 Barabási-Albert graphs, 5/5 Erdős-Rényi graphs, and 5/5 Watts-Strogatz graphs. It also used fewer BFS traversals on these families.

The Facebook graph showed the opposite behavior. On `facebook_combined`, both algorithms returned diameter 8, but Crescenzi finished in 0.029051 seconds using only 5 BFS traversals, while Takes-Kosters needed 0.060778 seconds and 10 BFS traversals. This suggests that Crescenzi's 4-Sweep high-degree initialization is especially effective on this real-world graph.

5.4 Discussion

The results suggest that Takes-Kosters scales more gracefully on the large synthetic families because its bound updates prune many candidates early and reduce the number of BFS traversals. Crescenzi, however, performs very well when a strong initialization quickly leads to tight bounds, which appears to be the case on the Facebook graph. Overall, Takes-Kosters is the better choice for the large synthetic benchmarks in this project, while Crescenzi is more effective on the tested real-world Facebook graph.

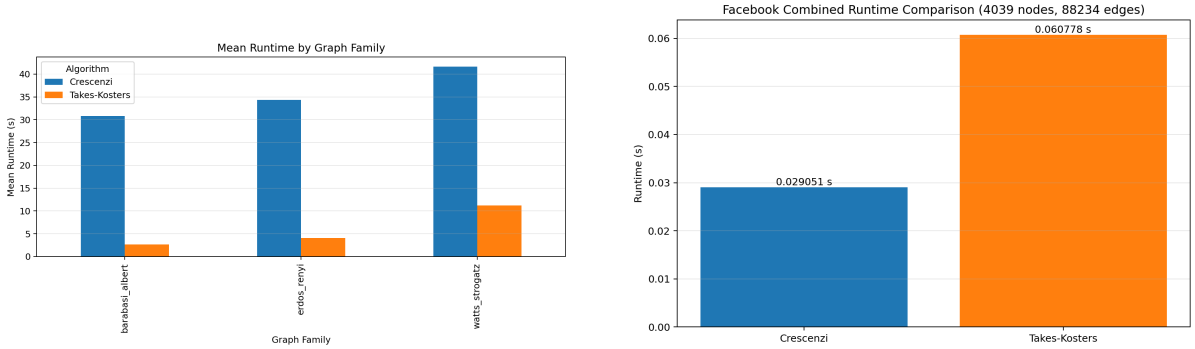
6 Comparison Plots



(a) Runtime vs. Nodes

(b) BFS Traversals vs. Nodes

Figure 1: Main performance comparison plots comparing runtime and BFS usage of the two algorithms across graph families.



(a) Runtime by Family

(b) Facebook Runtime Comparison

Figure 2: Additional comparison plots for family-wise behavior and the real-world Facebook graph.

7 Conclusion

This project compared Crescenzi's iFUB algorithm and the Takes-Kosters BoundingDiameters algorithm for graph diameter computation on one real-world graph and several synthetic graph families. The implementations behaved correctly in earlier validated small-graph experiments, where both algorithms matched the exact baseline. In the large-profile experiments used for the main comparison, the focus was therefore on scalability and practical efficiency.

The main conclusion is that Takes-Kosters is faster in practice on the large synthetic benchmark families and also uses fewer BFS traversals. It was faster on most Barabási-Albert graphs and on all tested Erdős-Rényi and Watts-Strogatz graphs, showing better scaling as graph size increases. This indicates that its pruning-based strategy is effective on large sparse synthetic networks.

At the same time, the Facebook graph produced a different outcome. Both algorithms returned diameter 8, but Crescenzi was faster and used only half as many BFS traversals as Takes-Kosters. This shows that graph structure plays an important role in practical performance, and that no single algorithm is uniformly best for every graph family.

Overall, the experiments answer the main project questions clearly: Takes-Kosters scales better and is usually faster on the synthetic families used here, while Crescenzi performs especially well on the tested real-world graph. Future work could include evaluating additional real-world datasets, extending the exact validation set, and studying how different initialization or candidate-selection strategies change the practical behavior of both algorithms.

References

[1] P. Crescenzi, R. Grossi, M. Habib, L. Lanzi, and A. Marino. *On Computing the Diameter of Real-World Undirected Graphs*. Theoretical Computer Science, 514:84–95, 2013. `https://www.sciencedirect.com/science/article/pii/S0304397512008687`

[2] F. W. Takes and W. A. Kosters. *Determining the Diameter of Small World Networks*. In Proceedings of the 20th ACM International Conference on Information and Knowledge Management (CIKM 2011), pages 1191–1196, 2011. `https://dl.acm.org/doi/epdf/10.1145/2063576.2063748`

[3] J. Leskovec and A. Krevl. *SNAP Datasets: Stanford Large Network Dataset Collection*. `https://snap.stanford.edu/data/`

[4] J. McAuley and J. Leskovec. *Learning to Discover Social Circles in Ego Networks*. In Advances in Neural Information Processing Systems 25 (NIPS 2012), pages 539–547, 2012. `https://snap.stanford.edu/social2012/papers/mcauley-leskovec.pdf`